

Suite au développement réussi du module de gestion des approvisionnements et des stocks pour l'entreprise Tricol, la direction informatique souhaite maintenant **sécuriser l'accès à cette application avant sa mise en production**.

L'application gère des **données sensibles** :

- Informations fournisseurs et contrats commerciaux
- Prix d'achat négociés
- Valorisation des stocks
- Historique des commandes et transactions financières

Ces données doivent être **accessibles à différents profils d'utilisateurs** au sein de l'entreprise, avec des niveaux d'accès différenciés selon les responsabilités de chacun.

De plus, dans une démarche **DevOps moderne**, l'entreprise souhaite mettre en oeuvre un processus de développement et de déploiement pour :

- Garantir la qualité du code
- Automatiser les tests et le déploiement
- Faciliter la mise en production
- Assurer la traçabilité des modifications

Utilisateurs et Rôles:

L'application dispose de 4 rôles distincts. le fichier Matrice des Permissions ci-joint pour le détail complet des autorisations par rôle.

Objectifs du Projet

Objectifs de Sécurité

Implémenter un système d'authentification et d'autorisation permettant de :

Authentifier les utilisateurs via deux méthodes complémentaires :

- Authentification locale (utilisateurs internes)
- Authentification déléguée via OAuth2/Keycloak (serveurs externes, SSO)

Gérer les autorisations de manière dynamique :

- Attribution de rôles métier aux utilisateurs
- Personnalisation des permissions au niveau individuel
- Adaptation des droits selon les besoins opérationnels

Protéger les ressources de l'application :

- Sécurisation des endpoints REST selon les rôles et permissions
- Contrôle d'accès aux données sensibles

Tracer les activités :

- Journalisation de toutes les actions sensibles
- Historique des connexions et modifications de permissions

Configurer la validation CORS pour limiter les origines approuvées.

Objectifs DevOps

Mettre en place une infrastructure complète permettant de :

- **Conteneuriser l'application**
- **Orchestrer l'infrastructure**
- **Automatiser le cycle de développement** :
 - Intégration continue (build, tests, analyse)
 - Déploiement automatisé
 - Validation de la qualité du code
- **Garantir la qualité** :
 - Analyse statique du code
 - Détection des vulnérabilités et bugs
 - Mesure de la couverture de tests

Exigences Techniques – Infrastructure

1. Conteneurisation avec Docker

L'application et tous ses composants doivent être conteneurisés pour garantir la portabilité et la reproductibilité.

2. Orchestration avec Docker Compose

L'infrastructure complète doit être déployable via un **fichier docker-compose.yml unique**.

3. Pipeline CI avec Jenkins

Un pipeline d'**intégration et déploiement continus** doit être mis en place pour automatiser le cycle de vie de l'application.

4. Analyse de Qualité avec SonarQube

Une plateforme d'**analyse de qualité** doit être configurée pour garantir la maintenabilité et la sécurité du code.

Exigences de Tests

Tests Unitaires

Couverture minimale requise : 80%

A. Tests d'Authentification et d'Autorisation

Tests liés au serveur d'autorisation (Get Access Token API) :

Test 1 : testGetAccessTokenFail

- **Objectif** : Vérifier que le serveur renvoie une erreur lorsque des identifiants incorrects sont fournis
- **Scénario** :
 - Envoyer une requête au serveur d'autorisation avec un Client ID ou Client Secret incorrect
 - Vérifier le statut HTTP renvoyé : **401 Unauthorized**
 - Vérifier le message d'erreur : "error": "invalid_client"

Test 2 : testAccessTokenSuccess

- **Objectif** : Confirmer que l'API renvoie un token valide avec des identifiants corrects
- **Scénario** :
 - Envoyer une requête au serveur d'autorisation avec des identifiants corrects (Client ID et Client Secret valides)
 - Vérifier le statut HTTP : **200 OK**
 - Vérifier la présence d'un document JSON contenant :
 - access_token : Jeton d'accès valide (non vide)
 - token_type : Type de jeton (doit être "Bearer")
 - expires_in : Délai d'expiration en secondes

Tests liés au serveur de ressources (Product APIs) :

Test 3 : testListProductWithPermissionRead

- **Objectif** : Vérifier que l'accès à la liste des produits est autorisé avec la permission "read"

- **Scénario :**
 - Simuler un token JWT avec la permission **read**
 - Effectuer une requête **GET /api/products**
 - Vérifier le statut HTTP :
 - **200 OK** si des produits existent
 - **204 No Content** si la liste est vide

Test 4 : testProductWithPermissionRead

- **Objectif :** Vérifier la lecture d'un produit spécifique avec la permission **read**
- **Scénario :**
 - Créer un produit de test en base de données
 - Simuler un token JWT avec la permission **read**
 - Effectuer une requête **GET /api/products/{id}**
 - Vérifier le statut HTTP : **200 OK**
 - Vérifier que le JSON retourné contient les détails du produit

Test 5 : testAddProductWithPermissionWrite

- **Objectif :** Valider que l'ajout d'un nouveau produit réussit avec la permission **write**
- **Scénario :**
 - Simuler un token JWT avec la permission **write**
 - Effectuer une requête **POST /api/products** avec un JSON de produit valide
 - Vérifier le statut HTTP : **201 Created**
 - Vérifier que le produit a bien été créé en base de données

Test 6 : testAddProductWithPermissionRead

- **Objectif :** Vérifier que la tentative d'ajout d'un produit est refusée avec seulement la permission **read**
- **Scénario :**
 - Simuler un token JWT avec **uniquement** le scope **read** (pas de permission **write**)
 - Effectuer une requête **POST /api/products** avec un JSON de produit valide
 - Vérifier que la requête est refusée avec le statut HTTP : **403 Forbidden**
 - Vérifier le message d'erreur indiquant l'insuffisance de droits

Technologies et Stack Technique

Backend

- Sécurité : **Spring Security 6**
- OAuth2 : **Spring Security OAuth2 Resource Server**
- Base de données : **PostgreSQL / MySQL**
- ORM : **Spring Data JPA / Hibernate**
- authentification stateless avec **JWT**
- Validation : **Hibernate Validator**
- Tests : **JUnit 5, Mockito**
- Documentation API: **Swagger**

Infrastructure

- Conteneurisation : **Docker**
- Orchestration : **Docker Compose**
- Serveur Auth : **Keycloak**
- CI/CD : **Jenkins**
- Qualité : **SonarQube**